UNIVERSIDAD POLITÉCNICA DE MADRID

E.T.S DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROYECTO FIN DE GRADO

INGENIERÍA DEL SOFTWARE



SISTEMA DE RECUPERACIÓN DE INFORMACIÓN BASADO EN RAG APLICADO A TRABAJOS DE FIN DE GRADO DE LA ETSISI

Desarrollado por: HUGO BLANCO IRAOLA

Dirigido por: ELVIRA AMADOR DOMINGUEZ

Junio 2025

Resumen

En el presente Trabajo de Fin de Grado, en adelante TFG, se desarrolla un sistema de recuperación de la información basándose en la técnica de Retrieval-Augmented Generation (RAG) que se centra en la búsqueda eficiente y veloz de los Trabajos de Fin de Grado defendidos en la ETSISI de la Universidad Politécnica de Madrid. Retrieval-Augmented Generation (RAG) es una técnica dentro del campo del Procesamiento del Lenguaje Natural (PLN) que une la recuperación de información con la generación de texto para ser capaz de responder a cualquier pregunta usando datos externos, en este caso, los TFGS defendidos en la Escuela. Esta integración permite generar respuestas más contextualizadas y precisas, superando las barreras de los modelos generativos tradicionales. El desarrollo de este sistema permite realizar consultas sobre estos TFGs mediante lenguaje natural, mejorando la accesibilidad a la información y ayudando a la localización de esta para estudiantes, investigadores, etc.

El desarrollo de este sistema se ha dividido en tres fases. En primer lugar, se ha comenzado con la extracción de la información y el procesamiento de los datos de los TFGs ya defendidos y publicados en la página web de archivos digitales de la Universidad Politécnica de Madrid, oa.upm.es. Esta fase se ha llevado a cabo mediante web scrapping con Selenium, consiguiendo extraer esta información y estructurándola en un archivo JSON. A continuación, con la implementación de un modelo del servidor de Ollama, “Stablelm2” y la biblioteca LangChain se ha implementado el sistema RAG. La elección de este modelo se basa en la capacidad de ser modelo generativo enfocado en QA y en español.

Finalmente, se evaluó el sistema mediante las métricas BERTScore, que nos permite medir de manera clara y objetiva la calidad de las respuestas generadas por el modelo contrastándolas con sus respuestas reales. Gracias a esta evaluación comprobamos que el sistema responde de manera relevante y coherente a las distintas consultas realizadas en lenguaje natural, demostrando que es capaz de acceder a la información guardada de los TFGs de una manera eficaz e intuitiva.

En cuanto a los resultados, el sistema es capaz de hacer búsquedas precisas sobre la información alojada en el JSON devolviendo una respuesta generada por el modelo en lenguaje natural. El sistema es robusto y escalable ya que ha sido sometido a una batería de pruebas cuando la muestra de información del JSON era escasa y cuando esta contaba con la información de hasta 30 TFGs distintos. En ambos casos el modelo responde correctamente a las preguntas realizadas y no sufre “alucinaciones” al aumentar la cantidad de información a procesar.

Como conclusión, este sistema representa un gran avance en la accesibilidad a este tipo de información con técnicas de Inteligencia Artificial y Procesamiento del Lenguaje Natural. Como futuras mejoras se ha planteado la integración de alguno modelo más grande que sea capaz de soportar más información con la finalidad de crear una herramienta, robusta y escalable, para que cualquier usuario pueda tener un acceso más rápido e intuitivo a esta gran cantidad de información.

Abstract

This Final Degree Project presents the development of an information retrieval system based on the Retrieval-Augmented Generation (RAG) technique, focused on the efficient and fast search of Final Degree Projects defended at the ETSISI of the Universidad Politécnica de Madrid. Retrieval-Augmented Generation (RAG) is a technique within the field of Natural Language Processing (NLP) that combines information retrieval with text generation to answer any question using external data, in this case, the TFGs defended at the University. This integration enables the generation of more contextualized and accurate responses, overcoming the limitations of traditional generative models. The developed system allows users to query these TFGs using natural language, improving access to academic information for students, researchers and other interested users.

The development of the system has been divided into three phases. First, the information was extracted and processed from already published TFGs available on the university's digital archive website (oa.upm.es). This phase was carried out using web scrapping with Selenium, successfully extracting the data and structuring it into a JSON file. Then, the RAG system was implemented using the "StableLM2" model deployed via the Ollama server and the LangChain library. This model was chosen for its ability to generate responses in Spanish and its optimization for question-answering task.

Finally, the system was evaluated using BERTScore metrics, which allow for an objective and accurate measurement of the quality of the generated responses by comparing the with ground truth answers. This evaluation confirmed that the system provides coherent and relevant answers to natural language queries, demonstrating its ability to effectively and intuitively retrieve the stored TFG information.

In terms of results, the system is capable of conducting accurate searches on the JSON-stored information and returning responses generated in natural language. The system is both robust and scalable, as it was tested with small data samples as well as with datasets containing up to 25 different TFGs. In all cases, the model responded correctly to the queries without generating "hallucinations" as the amount of information increased.

In conclusion, this system represents a significant advancement in accessing academic information through Artificial Intelligence and Natural Language Processing techniques. As a future improvement, the integration of a more powerful model is proposed to support larger volumes of data, with the goal of creating a robust and scalable tool that allows any user to quickly and intuitively access a large collection of academic documents.

Agradecimientos

Quiero empezar expresando mi más profundo agradecimiento a Laura Navalón, mi novia y compañera de vida, por estar a mi lado en cada etapa de este camino. Su apoyo constante, comprensión y ánimo incondicional han sido la clave para superar cualquier obstáculo. Gracias por acompañarme en lo malos momentos, por celebrar cada pequeño logro conmigo y por recordarme por qué vale la pena esforzarse. Este, y todos mis logros, son por y para ti.

Mi más sincero agradecimiento a Elvira Amador, mi tutora, por su comprensión, cercanía y la implicación que ha demostrado durante el desarrollo de este proyecto. Su orientación experta, sus precisos consejos y su disposición constante para ayudar han sido determinantes para dar forma a este trabajo. Gracias por enseñarme tanto, no solo sobre lo relacionado con el proyecto, sino también con cómo afrontar este tipo de retos académicos con rigor y responsabilidad.

A mi familia, por ser siempre mi base, mi refugio y mi ejemplo. Gracias por confiar siempre en mí, por apoyarme incondicionalmente y por hacerme sentir que no camino solo. Este logro es fruto de vuestros valores y enseñanzas.

Y por supuesto, a mis amigos, por aportar siempre ese toque de humor tan necesario en los momentos de agobio, por escucharme y por recordarme que, aunque estemos creciendo, seguimos caminando juntos.

Índice

1. Introducción 9
   INTRODUCCIÓN 9
   MOTIVACIÓN 9
   OBJETIVO 9
   ALCANCE 9
2. Estado de la técnica y contexto tecnológico 11
   2.1 Estado de la técnica 11
      Búsqueda Semántica 11
      Inteligencia Artificial Generativa 14
      Large Lenguage Models (LLMs) 15
      Retrieval-Augmented Generation (RAG) 16
      Bases de Datos Vectoriales 16
   2.2 Trabajos Previos 17
   2.3 Contexto tecnológico 18
      Herramientas y librerías utilizadas 18
      Tecnologías evaluadas y descartadas 19
   2.4 Conclusión 19
3. Desarrollo 21
   3.1 Módulo de scrapping 21
   3.2 Módulo principal 27
   3.3. Módulo de evaluación 30
      Conjuntos de evaluación 31
4. Evaluación y resultados 32
   4.1 Diseño de la experimentación 32
      Procedimiento 33
   4.2 Resultados obtenidos 33
   4.3 Conclusiones 37
5. Conclusiones y trabajos futuros 38
   5.1 Conclusiones 38
   5.2 Trabajos futuros 38
6. Sostenibilidad e impacto social 40
Bibliografía 41

Índice de figuras

Figura 1 Funcionamiento Búsqueda Semántica KDnuggets (2025) 12
Figura 2 Funcionamiento de Word2Vec Mikolov, T., et al. (2013) 13
Figura 3 Esquema Arquitectura Transformers Vaswani, A., et al. (2017) 14
Figura 4 Arquitectura Sistema RAG IBM (2023) 16
Figura 5 Estructura de la solución propuesta 21
Figura 6 Ejemplo de un TFG extraído en formato JSON 27
Figura 7 Prompt empleado para interactuar con el usuario 29

1. Introducción

INTRODUCCIÓN

En la era digital actual, la producción de información académica crece a un ritmo acelerado. Sin embargo, el acceso eficiente a esa información sigue siendo un reto, especialmente cuando se trata de documentos específicos como los Trabajos de Fin de Grado (TFG). A pesar de la existencia de repositorios digitales como el de la Universidad Politécnica de Madrid (oa.upm.es), estos no suelen ofrecer mecanismos intuitivos o inteligentes para la recuperación de información, dificultando el acceso a contenidos relevantes por parte de estudiantes, investigadores y otros usuarios interesados.

Esta problemática pone de manifiesto la necesidad de soluciones más eficaces para la búsqueda de información académica. En particular, se ha identificado la falta de un sistema que permita realizar consultas sobre TFGs utilizando lenguaje natural, lo que facilitaría significativamente la búsqueda de información en el repositorio, la cual actualmente se realiza mediante búsquedas exactas por campos (título, autor, año, etc.). Este trabajo busca cubrir ese vacío mediante la implementación de un sistema de recuperación de información basado en la técnica **Retrieval-Augmented Generation (RAG)**, que combina modelos generativos con bases de conocimiento para ofrecer respuestas precisas y contextualizadas.

MOTIVACIÓN

La motivación para abordar este proyecto surge, por un lado, del deseo de aportar una herramienta útil y accesible que facilite la consulta de información académica a cualquier usuario, independientemente de su nivel técnico. Por otro lado, nace del interés personal en el campo de la Inteligencia Artificial y el Procesamiento de Lenguaje Natural (PLN), explorando su aplicación práctica en un contexto real con impacto directo en la comunidad académica.

OBJETIVO

El objetivo principal del proyecto es desarrollar un sistema capaz de responder preguntas en lenguaje natural sobre los TFGs defendidos en la Escuela Técnica Superior de Ingeniería de Sistemas Informáticos (ETSISI) de la Universidad Politécnica de Madrid. Para ello, se ha diseñado un flujo de trabajo dividido en varias fases: extracción y estructuración de los datos mediante técnicas de *scraping*, indexación semántica utilizando *embeddings*, e implementación de un modelo generativo utilizando la técnica RAG en un entorno local mediante Ollama. Además, el sistema ha sido evaluado con múltiples consultas para garantizar el correcto funcionamiento y que sea capaz de generar respuestas relevantes y coherentes en lenguaje natural.

ALCANCE

El alcance del proyecto se centra en la creación de una herramienta funcional que permita búsquedas precisas sobre un subconjunto de TFGs disponibles en el

repositorio digital de la UPM, sirviendo como prueba de concepto para una futura ampliación del sistema.

En cuanto a la estructura técnica del trabajo, el proyecto se divide en los siguientes módulos:

- **Capítulo 2: Estado de la técnica y contexto tecnológico**
- **Capítulo 3: Desarrollo**
- **Capítulo 4: Evaluación y resultados**
- **Capítulo 5: Conclusiones y trabajos futuros**
- **Capítulo 6: Sostenibilidad e impacto social**

Este Trabajo de Fin de Grado representa la unión entre una necesidad real, una motivación académica y personal, y la aplicación de tecnologías actuales para ofrecer una solución práctica en el ámbito de la recuperación de información.

2. Estado de la técnica y contexto tecnológico

2.1 Estado de la técnica

Este proyecto se enmarca en el campo del Procesamiento del Lenguaje Natural (PLN), una de las áreas más relevantes en los últimos años dentro de la Inteligencia Artificial. Esta área de investigación se centra en el desarrollo de técnicas y modelos que permitan a las máquinas comprender, generar e interpretar el lenguaje humano de una forma muy precisa.

Gracias a los últimos avances dentro del desarrollo de modelos de aprendizaje profundo, y a la aparición de arquitecturas como los *transformers*, se ha conseguido mejorar inmensamente la interacción entre los seres humanos y los sistemas inteligentes en diversas tareas como traducción automática y generación o recuperación de información.

Este proyecto se centra en aplicar técnicas de PLN para facilitar el acceso y la búsqueda de información de TFGs. A continuación, se explorarán en detalle conceptos y tecnologías utilizadas para el correcto funcionamiento del sistema como la búsqueda semántica, los modelos generativos, el uso de *embeddings* y los sistemas RAG.

Búsqueda Semántica

Desde sus inicios, los motores de búsqueda se han basado en el “match” literal de palabras, es decir, su objetivo era recuperar para el usuario los documentos que contuvieran más palabras clave, dando por hecho que esa sería la mejor respuesta a la consulta del usuario. Sin embargo, ya se ha demostrado que esa solución no es del todo eficiente.

Para solucionar este problema se creó el concepto de búsqueda semántica, Latard (2017). Esta técnica dentro del Procesamiento de Lenguaje Natural (PLN) y tiene por objetivo mejorar la precisión y relevancia de los resultados de búsqueda. Lo que se busca es entender el contexto de una frase o palabra clave para poder encontrar relaciones dentro de los documentos a devolver como respuesta.

Esta solución se ha convertido en una herramienta clave para consultas más complejas como puede ser la búsqueda de información en documentos legales, artículos científicos o información médica.

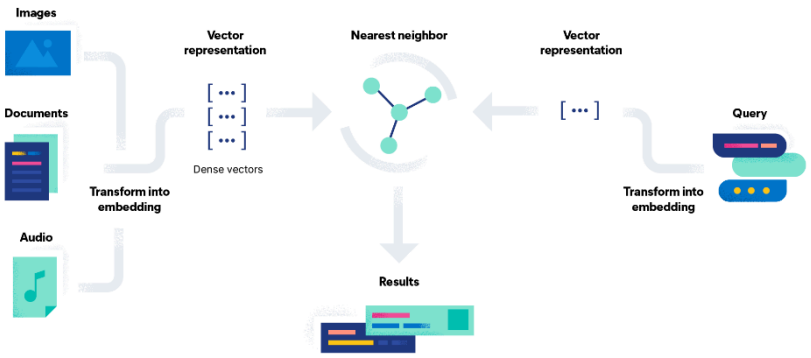


Figura 1 Funcionamiento Búsqueda Semántica KDnuggets (2025)

Las técnicas de búsqueda semántica se basan en algoritmos y modelos de PLN. Dentro de estas técnicas, podemos encontrar las llamadas técnicas de recuperación de información o information retrieval. Dentro de este tipo de técnicas, existen una serie de aproximaciones de búsqueda basadas en el uso de representaciones vectoriales. De esta manera, los textos se almacenan en la base de datos en forma de vector y mediante el uso de métricas de similitud vectorial se recuperan aquellos textos más parecidos a la consulta proporcionada. Una parte esencial de este proceso es la conversión de textos a formato vectorial. Unos de los primeros modelos en realizar esta tarea fueron Word2Vec Mikolov (2013), GloVe Jeffrey Pennington (2014) y Fastext Bojanowski (2017). Estos modelos epresentan las palabras de la consulta en un campo vectorial donde las palabras con contextos similares se encuentras más cerca unas de otras, ayudando a que el sistema pueda entender relaciones semánticas más complejas. Word2Vec fue el primer modelo de lenguaje, consiguiendo que, a través de estas representaciones vectoriales con el uso de redes neuronales, los sistemas pudieran entender la similitud semántica entre diferentes palabras.

El problema de estos modelos es que estas representaciones vectoriales, también llamados embeddings, generadas son estáticas, es decir, independientemente del contexto una palabra siempre está representada por el mismo vector. Esto da lugar a errores al usar palabras polisémicas, ya que estas varían su significado dependiendo del contexto. Debido a estos problemas surge la necesidad de crear una solución para este comportamiento: los modelos contextuales.

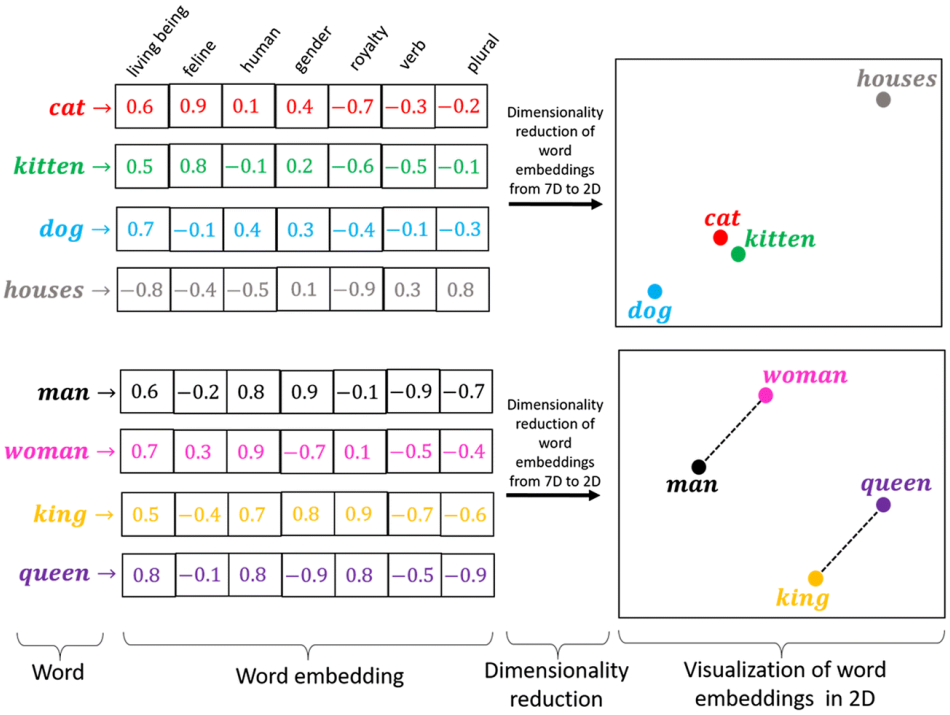


Figura 2 Funcionamiento de Word2Vec Mikolov, T., et al. (2013)

Los modelos contextuales poseen la capacidad de generar representaciones dinámicas de las palabras en función del contexto en el que se encuentren. Para ello generan embeddings que almacenan el significado de una palabra en relación con el resto de las palabras del contexto. Para crear estos modelos se introdujo la arquitectura *Transformers*. **(Vaswani et al., 2017)**

La arquitectura Transformers se basa en el mecanismo de self-attention que consiste en evaluar la importancia de cada palabra en un contexto con respecto al resto de palabras, capturando y generando así las relaciones entre ellas.

Dentro de esta arquitectura encontramos modelos clave como puede ser BERT Devlin (2019), son modelos de lenguaje pre-entrenado que usa esta arquitectura para procesar texto de manera bidireccional. Esta capacidad de entender el contexto es clave para las tareas que requieren una comprensión profunda del lenguaje.

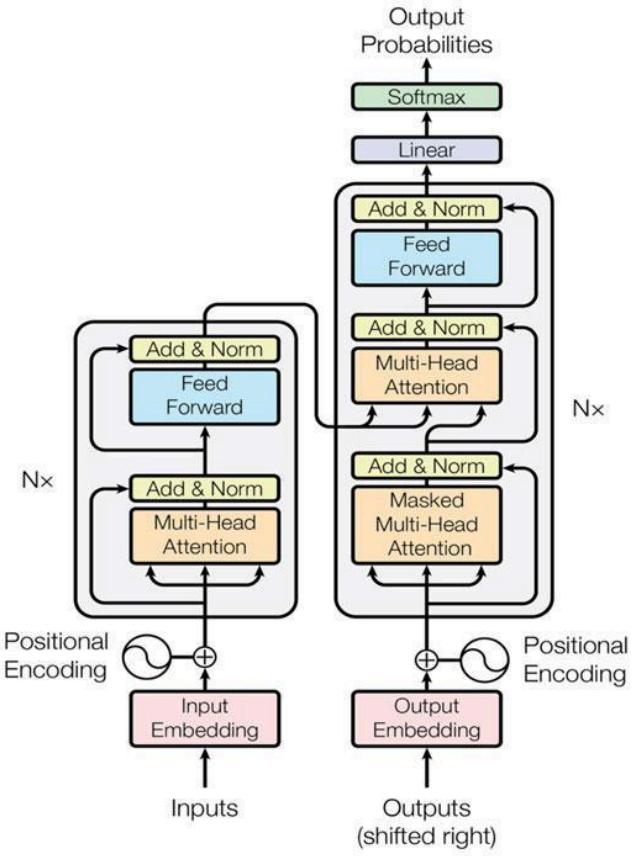


Figura 3 Esquema Arquitectura Transformers Vaswani, A., et al. (2017)

Inteligencia Artificial Generativa

La inteligencia artificial generativa es un conjunto de métodos diseñados para generar contenido original y nuevo a partir de unos datos iniciales. Aplicándolo al Procesamiento de Lenguaje Natural, estos modelos son capaces de generar texto coherente y relacionadocon el contexto.

Anteriormente usaban reglas gramaticales y formales para producir lenguaje natural pero estas técnicas estaban muy limitadas, impidiendo crear contenido complejo y flexible. Gracias al desarrollo del aprendizaje profundo (deep learning) la capacidad de la IA Generativa aumentó considerablemente. Aun así, los primeros modelos no conseguían manejar volúmenes grandes de datos. Con el posterior desarrollo de las arquitecturas como LSTM y las GRU (Gated Recurrent Units, Cho (2014)) se aumentó su capacidad, mejorando la generación de texto.

Las mejoras recientes en IA Generativa han sido impulsadas por la introducción de la arquitectura Transformers, previamente mencionada. Dentro de esta estructura Transformers podemos destacar un modelo clave a la hora de la generación de texto, GPT (Generative Pre-Trained Transformer). **(Gadekallu et al., 2023)**

GPT es una arquitectura de IA Generativa que ha hecho historia dentro del mundo del Procesamiento de Lenguaje Natural, es un modelo capaz de generar texto coherente únicamente con breves indicaciones textuales, denominadas prompt. Gracias a la arquitectura Transformers, el modelo es capaz de aprender patrones y contextos lo

que le ayuda a producir lenguaje natural de una forma más compleja y avanzada. Dentro de la familia de GPT encontramos diferentes versiones, la más reciente es GPT-4o OpenAI (2023) la cual ha mejorado enormemente la calidad y coherencia del texto generado, aumentando el desempeño en diversas tareas como asistencia virtual o chatbots.

Actualmente se han desarrollado modelos multi-modales que integran diferentes tipos de datos para realizar tareas complejas. Destacan aplicaciones en como la generación de imágenes mediante prompts de texto con modelos como DALL-E, Ramesh (2021) de OpenAI. Estos modelos multi-modales suponen un gran avance sobre los modelos uni-modales ya que consiguen una mejor generación de contenido, una interacción más fluida con los usuarios y una mayor capacidad del modelo para interpretar y procesar los datos recibidos.

Large Lenguage Models (LLMs)

El lenguaje es una herramienta fundamental dentro de la comunicación entre los seres humanos, la usamos desde nuestro nacimiento de diversas formas. Esta capacidad de comunicación mediante lenguaje humano no la tienen las máquinas, no son capaces de comprender y comunicarse entre ellas o con los seres humanos sin la ayuda de herramientas de Inteligencia Artificial (Zhao et al., 2023a). En los últimos años, uno de los mayores retos científicos ha sido dotar a las máquinas con habilidades similares a las humanas en campos como la escritura, comunicación y lectura (Turing, 2009).

El uso del aprendizaje profundo (Deep Learning) con grandes recursos computacionales y volúmenes de datos han conseguido crear los Large Lenguaje Models, de ahora en adelante LLM. Estos modelos son redes neuronales de gran escala entrenadas con enormes cantidades de texto no etiquetadas mediante la técnica de aprendizaje autodirigido (self-supervised learning) (Shen et al., 2023). Gracias a estos entrenamientos son capaces de aprender patrones complejos y relaciones semánticas en los datos de entrenamiento.

Los LLM han obtenido magníficos resultados a la hora de realizar tareas de Procesamiento de Lenguaje Natural (o PLN por sus siglas), como pueden ser la traducción automática, síntesis de texto, respuesta a preguntas... El desarrollo más significativo se produjo al introducir la arquitectura *Transformers* (Vaswani, 2017), previamente comentada y explicada.

El pipeline de entrenamiento de los LLM consiste en recopilar datos de texto proveniente de diversas fuentes, estos datos son procesados y limpiados y, finalmente, el modelo se entrena a través de un proceso iterativo. Dentro de este proceso se incluye la inicialización aleatoria de parámetros y la optimización progresiva de los mismos con técnicas de Deep Learning. Tras ser entrenado, el LLM es capaz de ofrecer los distintos servicios a tareas de PLN.

Los últimos estudios e investigaciones con LLMs han demostrado ser muy útiles en campos como la medicina y la salud (Khare et al., 2021), así como en análisis de textos relacionados con política, leyes u otras ciencias sociales (Liu et al., 2021).

Todas estas aplicaciones demuestran la versatilidad y la capacidad de adaptabilidad de los LLMs a cualquier contexto.

Retrieval-Augmented Generation (RAG)

Las técnicas de Inteligencia Artificial han mejorado y evolucionado en los últimos años, consiguiendo una mejor recuperación y generación de información. Aun así, los modelos generativos tienen la limitación de que su conocimiento se limita a la información procesada durante su entrenamiento por lo que en caso de recibir una consulta fuera de este campo el modelo no sería capaz de generar una respuesta coherente. Para solucionar este problema sería necesario reentrenar al modelo con nueva información, siendo una solución poco eficiente. Por otro lado, los modelos de recuperación de información se basan en la búsqueda de documentos relevantes, teniendo la limitación de que no son capaces de sintetizar dichos documentos para extraer solo la información necesaria y relevante con la consulta. Gracias a estas limitaciones surge el RAG (Retrieval-Augmented Generation), que busca combinar ambas aproximaciones para dar solución a ambos problemas.

El RAG es una técnica que mejora los LLMs (Large Lenguage Models) añadiendo conocimientos de fuentes externas, es decir, es capaz de obtener información relevante y específica y, a su vez, de generar una respuesta coherente y alineada con la consulta realizada por el usuario. Este nuevo desarrollo ha sido clave para mantener la relevancia y precisión de chatbots u otras aplicaciones similares asegurando que el usuario recibe una información lo más exacta posible.

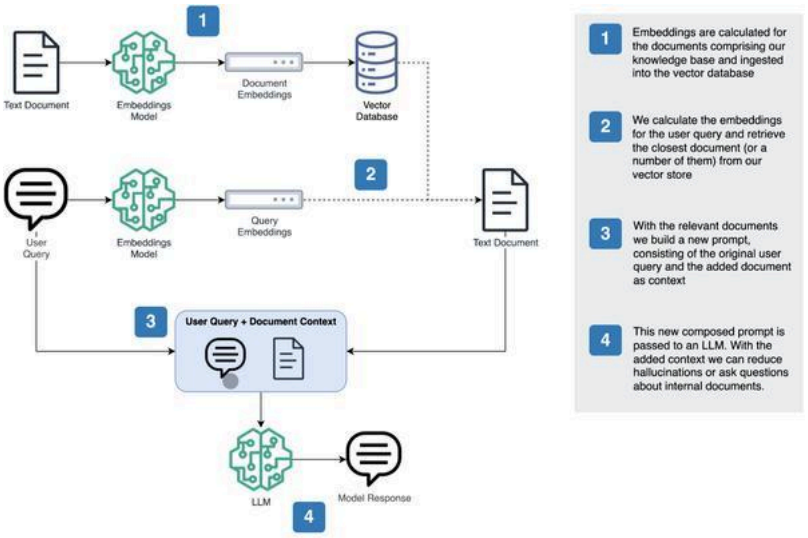


Figura 4 Arquitectura Sistema RAG IBM (2023)

Bases de Datos Vectoriales

Tras todos los avances en las metodologías y tecnologías relacionadas con la Inteligencia Artificial y la aparición de los sistemas RAG surge un nuevo inconveniente, cómo almacenar los datos no estructurados. Las bases de datos tradicionales tanto relacionales como NoSQL manejan bien los datos estructurados o semi-estructurados pero no son capaces de manejar datos como texto,audio, imágenes, etc. donde son

necesarias las comparaciones por similitud. Es por eso por lo que surgen las Bases de Datos Vectoriales (oVecDBs, por sus siglas en inglés).

Estos sistemas ofrecen las capacidades de las bases de datos tradicionales como son la optimización de consultas, mecanismos de privacidad y seguridad, escalabilidad y tolerancia a los fallos, pero para datos no estructurados. Este tipo de bases de datos son más eficientes, al reducir los costes de uso con cachés semánticas. Estas cachés permiten comprobar la relación de la consulta con otras previas para evitar enviarla al LLM, limitando ejecuciones redundantes y costosas.

La diferencia con los datos estructurados es que para recuperar datos no estructurados se emplea la búsqueda por similitud, no consultas SQL. Para poder realizar esta búsqueda es necesario que, previamente, se codifiquen los datos en vectores de D dimensiones mediante un modelo de *embeddings* para poder almacenarlos en las Bases de Datos Vectoriales.

2.2 Trabajos Previos

La implementación de sistemas RAG en entornos y proyectos académicos se basan en investigaciones que han demostrado su eficacia para mejorar la precisión en modelos generativos. El trabajo seminal de **Lewis et al., 2020** estableció el marco teórico para introducir mecanismos de recuperación de información en arquitecturas generativas, consiguiendo unas mejoras del 23% en la exactitud factual contra los modelos convencionales en tareas de QA (question answering) abierto. Este tipo de avances permitieron conseguir crear adaptaciones para el ámbito académico donde la precisión documental es crítica.

En cuanto a trabajos más específicos de gestión documental universitaria, estudios como el de **Gao et al. (2024)** han implementado este tipo de sistemas RAG para realizar consultas en repositios institucionales, consiguiendo un 89% de precisión en la recuperación de información mediante embeddings contextuales y estrategias de chunking semántico. A su vez, **Mezhlumyan (2024)** desarrolló un pipeline que combinaba GROBID con modelos Llama-2 reduciendo un 40% las alucinaciones del modelo en repuestas sobre contenido de papers académicos.

Thulke et al. (2021) propusieron un sistema de verificación multietapa que integraba RAG con clasificadores de consistencia lógica, reduciendo errores en un 67% respecto a implementaciones anteriores. Estos descubrimientos coinciden con las mejoras implementadas por **Asai et al. (2023)** con la recuperación cross-documental para reducir sesgos en fuentes únicas.

En las bases de datos vectoriales, las investigaciones de **Johnson et al. (2022)** optimizaron el IVF (Inverted File Index) y PQ (Product Quantization) para manejar vectores de hasta 20248 dimensiones en repositorios académicos. Más adelante, **Singlestore (2024)** demostró la indexación híbrida (gráfica y vectorial) con mejoras de un 32% en la velocidad de consultas semánticas complejas sobre corpus de más de cien mil documentos.

2.3 Contexto tecnológico

Herramientas y librerías utilizadas

Para el correcto desarrollo de este sistema se han utilizado diversas tecnologías, seleccionando las más compatibles con el correcto rendimiento y compatibilidad con el objetivo del proyecto. A continuación, se explican dichas herramientas con el motivo de su uso:

1.1. Lenguaje de Programación:

- Python: elegido por su facilidad a la hora de usarlo y combinarlo con diversas bibliotecas clave en el Procesamiento de Lenguaje Natural (PLN) y la Inteligencia Artificial (IA)

1.2. Web Scrapping

- Selenium: usado para la extracción de los datos procedentes de la página de la universidad, oa.upm.es, permitiendo obtener los metadatos necesarios de los TFGs publicados.
- BeautifulSoup: utilizado para el procesamiento y estructuración de los datos extraídos del HTML de la página web.
- Requests: nos permite realizar peticiones HTTP para evitar bloqueos en la página web con el uso de tiempos de espera aleatorios y la manipulación de las cabeceras.

1.3. Procesamiento y almacenamiento de Datos

- JSON: usado como almacenamiento de los datos extraídos en el scrapping.
- LangChain:
    - ChromaDB: base de datos vectorial usada para almacenar y recuperar documentos de manera rápida y eficiente.
    - HuggingFace Embeddings: modelo de embedding utilizado para representar semánticamente los textos, ayudando a mejorar la precisión a la hora de recuperar la información almacenada.
    - RecursiveCharacterTextSplitter: divide los textos en fragmentos adecuados para que se puedan indexar de una manera más sencilla.

1.4. Modelos de Lenguaje y Generación de Respuestas

- Ollama: servidor utilizado para la ejecución local del modelo de lenguaje.
- StableLM2: modelo de lenguaje generativo en español utilizado para responder preguntas en lenguaje natural. Es capaz de procesar la preguntar en lenguaje natural y generar una respuesta también en lenguaje natural.
- LangChain PromptTemplate y ChatPrompTemplate: herramientas usadas para la construcción de los prompts que ayudan al modelo a procesar la pregunta y poder generar una respuesta precisa y coherente.

1.5. Procesamiento de Preguntas y Respuestas

- Transformers de Hugging Face

- AutoTokenizer y AutoModelForQuestionAnswering: inicialemnte usados para evaluar los modelos de Pregunta-Respuesta basados en BERT, finalmente fueron descartados de cara a usar un modelo generativo más flexible.

1.6. Evaluación del sistema con métricas automáticas

- Con el fin de validar que el sistema responde de forma precisa y coherente a las diferentes consultas, se ha realizado una fase de evaluación con la métrica **BERTScore**. Evalúa el contenido semántico, permitiendo medir si la respuesta, aunque esté expresada con otras palabras, da la misma información. Se ha usado la librería bert-score y un modelo preentrenado bert-base-uncased, calculando así la precisión (P), recall (R) y F1-score como indicadores del rendimiento del sistema.

Tecnologías evaluadas y descartadas

Durante el desarrollo de este sistema se han evaluado diferentes opciones, decantándose por las óptimas para el correcto desarrollo y funcionamiento del proyecto.

- **Uso de modelos tipo BERT para preguntas y respuestas (QA):** se inició el desarrollo con modelos basados en BERT en español para responder preguntas sobre la información almacenada de los TFG. Sin embargo, este enfoque tenía limitaciones en cuanto a flexibilidad ya que no son modelos generativos y no era capaz de dar una respuesta en lenguaje natural de una forma efectiva. Finalmente se optó por StableLM2, un modelo dentro de Ollama que si es capaz de responder estas preguntas sobre la información almacenada generando una respuesta en lenguaje natural.
- **Indexación basada únicamente en texto sin embeddings:** el primer enfoque fue usar una búsqueda simple basada en palabras clave, pero fue descartada por el uso de embeddings semánticos con HuggingFace Embeddings, permitiendo mejorar la precisión en la recuperación de la información.
- **Almacenamiento persistente de los datos en una base de datos relacional:** Se evaluó la posibilidad de almacenar los TFGs en una base de datos relacional como MySQL o PostgreSQL, sin embargo, con ChromaDB obtenemos una mayor eficiencia a la hora de realizar búsquedas semánticas en textos extensos como pueden ser los resúmenes de los TFG.

2.4 Conclusión

El análisis del estado de la técnica y del contexto tecnológico ha brindado una base sólida para el correcto desarrollo del sistema propuesto. Se ha explicado cómo la evolución de la búsqueda semántica, los avances en la inteligencia artificial generativa y los modelos de lenguaje (LLMs) han creado nuevas maneras de interactuar con la información que nos rodea de una forma más natural, eficiente e intuitiva.

La integración de los sistemas RAG (Retrieval-Augmented Generation) han permitido combinar los puntos fuertes de la recuperación de información con la generación de respuestas en lenguaje natural. Gracias a esto, se reducen las limitaciones tradicionales de los modelos generativos, alucinaciones o falta de actualización, y se ofrece una solución más precisa y escalable para los sistemas de búsqueda especializados, como el desarrollado en este proyecto.

El ecosistema tecnológico utilizado ha sido clave a la hora de poder construir un sistema robusto y adaptable. Además, la elección de cada tecnología se ha basado en pruebas empíricas que han permitido descartar las opciones menos adecuadas para el correcto funcionamiento del proyecto.

En definitiva, se ha demostrado que el proyecto cuenta con un contexto tecnológico puntero, con las últimas novedades en procesamiento de lenguaje natural capaz de abordar un problema real dentro del ámbito académico: la recuperación de información relativa a los TFGs de una forma rápida, precisa y, sobre todo, intuitiva. No solo se ha conseguido desarrollar un sistema funcional, sino que también se abre la puerta a futuras mejoras e implementaciones más avanzadas y con más recursos disponibles.

3. Desarrollo

El sistema desarrollado se compone de tres módulos principales, el módulo de **scrapping, el proceso principal (main) y la evaluación**, organizados y estructurados para dividir responsabilidades y manejar de manera correcta la escalabilidad de este. El sistema implementado en este proyecto, y todo el código fuente se puede encontrar en el siguiente repositorio de GitHub:

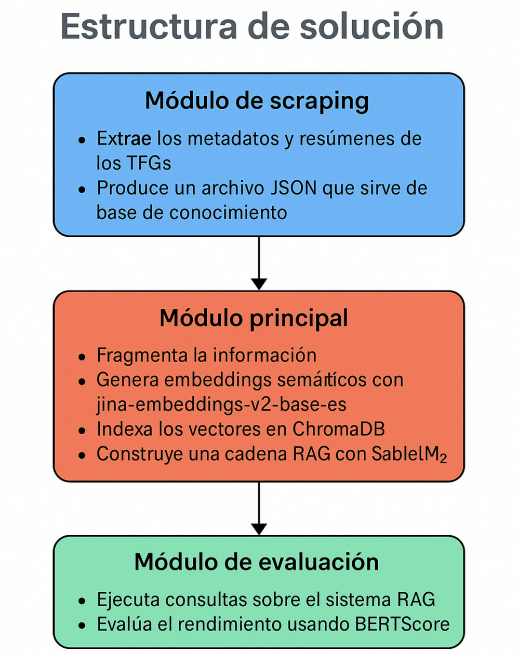


Figura 5 Estructura de la solución propuesta

3.1 Módulo de scrapping

El primer módulo desarrollado fue el módulo de ***Scrapping*** que es el encargado de extraer, de forma automatizada, la información del portal de la universidad Politécnica de Madrid, así como de convertirla a un formato estructurado para su posterior procesamiento.Para ser más exactos, se ha trabajado sobre la siguiente URL, filtrando por los TFGs defendidos en la **Escuela Técnica Superior de Sistemas Informáticos (ETSISI):**

https://oa.upm.es/cgi/search/archive/advanced?screen=Search&dataset=archive&title_merge=ALL&title=&creators_name_merge=ALL&creators_name=&contributors_name_merge=ALL&contributors_name=&abstract_merge=ALL&abstract=&keywords_merge=ALL&keywords=&subjects_merge=ANY&type=other&master_title_merge=ALL&master_title=&institution=ETSI_Sistemas_Infor&order=-date%2Fcreators_name%2Ftitle&_action_search=Buscar

Todo este proceso no ha sido algo mecánico ya que el contenido que devuelve la web no es una API ni una estructura de datos manejable, sino una página web construida

dinámicamente con HTML, con múltiples capas de etiquetas y estructuras anidadas difíciles de interpretar de una forma directa y clara.

Para el correcto funcionamiento de este módulo se ha utilizado Selenium, que actúa como un usuario por la web accediendo a los contenidos que se le indican, y BeatifulSoup, para parsear y extraer los datos del código HTML, automatizando la navegación y la conversión del contenido, generando un archivo JSON con los metadatos y resúmenes de los TFGs.

El trabajo de extracción de los datos del HTML en bruto y su conversión a un archivo estructurado en JSON plantea un gran reto, ya que es necesario identificar correctamente en el HTML aquellos elementos que nos permitan extraer de manera correcta y sistemática la información a buscar. A continuación, se muestran ejemplos de los fragmentos de HTML en bruto usados para extraer cada uno de los campos de un único TFG:

- *Título*

```
<div class="abstract_description">
  <h2>Descripción</h2>
  <table class="abstract_description ep_block citation table-citation-
  1">
    <tbody>
      <tr class="title">
        <th class="ep_row abstract_description eprint_fieldname_title">
        ...</th>
        <td class="ep_row abstract_description eprint_field_titl
        e"> == $0
          "Desarrollo e implementación de un brazo robótico articulado
          basado en el microcontrolador ESP8266"
        </td>
      </tr>
```

- *Autor*

```
<div class="abstract_description">
  <h2>Descripción</h2>
  <table class="abstract_description ep_block citation table-citation-1">
    <tbody>
      <tr class="title">…</tr>
      <tr class="creators">
        <th class="ep_row">
          "Autor/es"
          <span class="colon">:</span>
        </th>
        <td class="ep_row abstract_description eprint_fieldname_creators">
          <ul class="creators">
            <li class="creator creator-1"> == $0
              <span class="person_name">Gil Llorente, Sergio</span>
            </li>
          </ul>
        </td>
      </tr>
```

- *Director/es*

```
<th class="ep_row">
  "Director/es"
  <span class="colon">:</span>
</th>
<td class="ep_row abstract_description eprint_fieldname_contributors">
  <ul class="contributors">
    <li class="contributor contributor-1">
      <span class="person_name">Bordel Sánchez, Borja</span>
      <script language="JavaScript" type="text/javascript">

        genmail('88989borja.bordel@upm.es',

        '88988','1')
      </script>
      "("
      <a href="mailto:borja.bordel@upm.es">borja.bordel@upm.es
      </a>
      ") "
```

- *Fecha*

```
<tr class="date">
  <th class="ep_row"> == $0
    "Fecha"
    <span class="colon">:</span>
  </th>
  <td class="ep_row">Mayo 2025</td>
</tr>
```

- *Grado*

```
<tr class="grado">
  <th class="ep_row"> == $0
    "Grado"
    <span class="colon">:</span>
  </th>
  <td class="ep_row">Grado en Ingeniería de Computadores</td>
</tr>
```

- *Asignaturas*

```
<tr class="subjects">
  <th class="ep_row">
    "Materias"
    <span class="colon">:</span>
  </th>
  <td class="ep_row">
    <ul class="subjects">
      <li class="subject subject-1">
        <a href="https://oa.upm.es/view/subjects/informatica.html">Informática</a>
        ::after
      </li>
      <li class="subject subject-2"> == $0
        <a href="https://oa.upm.es/view/subjects/robotica.html">
        Robótica e Informática Industrial</a>
        ::after
      </li>
    </ul>
  </td>
</tr>
```

- *Institución*

```
<tr class="institution">
  <th class="ep_row">
    "Escuela"
    <span class="colon">:</span>
  </th>
  <td class="ep_row"> == $0
    <a href="https://oa.upm.es/view/institution/ETSI=5FSistemas=5FInfor/">E.T.S.I. de Sistemas Informáticos (UPM)</a>
  </td>
</tr>
```

- *Departamento*

```
<tr class="department">
  <th class="ep_row">
    "Departamento"
    <span class="colon">:</span>
  </th>
  <td class="ep_row"> == $0
    <a href="https://oa.upm.es/view/department/Sistemas=5Finformaticos=5F2014/">Sistemas Informáticos</a>
  </td>
</tr>
```

- *Objetivos de Desarrollo Sostenible (ODS)*

```
<tr class="ods">
  <th class="ep_row">…</th>
  <td class="ep_row">
    <ul class="ods">
      <li class="ods ods-1">
        <a href="https://oa.upm.es/view/ods/educacion=5Fcalidad/">
        04. Educación de calidad</a>
      </li>
      <li class="ods ods-2">
        <a href="https://oa.upm.es/view/ods/industria=5Finnovacion
        =5Finfraestructura/">09. Industria, innovación e
        infraestructura</a>
      </li>
      <li class="ods ods-3"> == $0
        <a href="https://oa.upm.es/view/ods/reduccion=5Fdesigualda
        des/">10. Reducción de las desigualdades</a>
      </li>
    </ul>
  </td>
</tr>
```

- *Palabras clave*

```
<tr class="keywords">
  <th class="ep_row"> == $0
    "Palabras Clave Informales"
    <span class="colon">:</span>
  </th>
  <td class="ep_row">Brazo robótico; Microcontrolador Esp8266;
  Arduino; API REST; Robótica</td>
</tr>
```

- *Derechos de uso*

```
<tr class="rights">
  <th class="ep_row"> == $0
    "Licencias Creative Commons"
    <span class="colon">:</span>
  </th>
  <td class="ep_row">Reconocimiento - Sin obra derivada - No
  comercial</td>
</tr>
```

- *Resumen*

```
<div class="ep_block abstract">
  <h2>Resumen</h2> == $0
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">Abstract:</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
  <p class="ep_field_para">...</p>
```

Cada campo ... corresponde a un párrafo tanto del resumen como del abstract.

- *Enlace PDF*

```
<table class="documents">
  <tbody>
    <tr>
      <td class="icon">
        <a href="https://oa.upm.es/88988/1/TFG_SERGIO_GIL_LLORENTE.pdf" class="ep_document_link">
          <img class="ep_doc_icon" alt="[thumbnail of TFG_SERGIO_GIL_LLORENTE.pdf]" src="https://oa.upm.es/style/images/fileicons/application_pdf.png" border="0">
        </a>
      </td>
      <td class="text">...</td> == $0
    </tr>
  </tbody>
</table>
```

Uno de los principales problemas encontrados durante el desarrollo del módulo fue el bloqueo a la web por tráfico sospechoso. Este problema ralentizó mucho el desarrollo del módulo, pero fue solventado mediante una función de peticiones robustas, “robust_request” dentro del código, que se encarga de introducir cabeceras HTTP variables con diferentes tiempos de espera entre estas peticiones. Con todo esto simulamos el comportamiento de un usuario real, reduciendo el riesgo de ser bloqueados y pudiendo realizar modificaciones en el código constantemente.

También fue imprescindible la implementación del control de navegación por páginas, ya que dentro del repositorio tenemos un número limitado de resultados por página. Para evitar este problema, se diseñó dentro del código que se pudiese iterar entre las páginas, accediendo al botón “Siguiente” de forma automática gracias a Selenium.

El resultado final del módulo son dos archivos JSON, tfgs.json y tfgs_pequeño.json, que contienen una lista de diccionarios en formato JSON con la siguiente estructura para cada TFG:


```
[
    {
        "título": "Diffusion Model Based Brain Tumour Segmentation Enhanced With Inpainting Method",
        "autor": "Cui, Che",
        "director": "Director/es:Díaz Álvarez, Albertohttps://orcid.org/0000-0002-4150-9052Iglesias Hernández
        "tipo_documento": "Tipo de Documento:Proyecto Fin de Carrera/Grado",
        "grado": "Grado:Grado en Ciencia de Datos e Inteligencia Artificial",
        "fecha": "Fecha:Marzo 2025",
        "asignaturas": "Materias:InformáticaMedicina",
        "ods": "ODS:03. Salud y bienestar09. Industria, innovación e infraestructura",
        "palabras_clave": "Palabras Clave Informales:Segmentación de tumor; Imágenes médicas; Supervisión déb
        "institución": "Escuela:E.T.S.I. de Sistemas Informáticos (UPM)",
        "departamento": "Departamento:Sistemas Informáticos",
        "derechos": "Licencias Creative Commons:Reconocimiento - Sin obra derivada - No comercial",
        "PDF_link": "https://oa.upm.es/88267/1/TFG_CHE_CUI.pdf",
        "resumen": "ResumenEn este trabajo se presenta un método para la segmentación de tumores cerebrales e
    },
```


Figura 6 Ejemplo de un TFG extraído en formato JSON

La versión tfgs_pequeño.json contiene la información de 6 TFGs y fue creada para realizar las primeras pruebas manuales tanto del código del Scrapping como para el correcto funcionamiento del modelo.

La versión tfgs.json contiene una muestra más representativa, información de 25 TFGs distintos, permitiendo evaluar el rendimiento y la escalabilidad del sistema.

Todo este proceso se ha llevado a cabo gracias a un estudio exhaustivo del HTML, múltiples pruebas que evitasen los bloqueos por parte del servidor y la validación manual de la información extraída, asegurando su consistencia y utilidad como base del sistema de recuperación de información.

Gracias al correcto desarrollo de este módulo se ha creado la base de conocimiento esencial que alimenta la arquitectura RAG del proyecto, permitiendo que las consultas se realicen sobre datos reales y estructurados con gran precisión.

3.2 Módulo principal

El módulo **principal o *main***, es el núcleo del sistema RAG y el encargado de la implementación completa del pipeline de este sistema. Aquí es donde se lleva a cabo la transformación de los datos extraídos por el módulo Scrapping para que sean accesibles y legibles por el modelo generativo. Este módulo sigue una lógica de fases que permite pasar de un conjunto de TFGs en formato JSON a un sistema capaz de

responder consultas en lenguaje natural sobre estos datos. A continuación, se describen en detalle las diferentes fases de este proceso.

A) Ingesta y fragmentación de los datos

La función ingest() recibe el archivo JSON con la información extraída de los TFG por el módulo Scrapping. Esta función interpreta los documentos, los convierte a formato Document de LangChain e inicia la preparación para el procesamiento semántico.

Con la ayuda de RecursiveCharacterTextSplitter manejamos los fragmentos de texto con los siguientes parámetros:

- Chunk_size = 1024
- Chunk_overlap = 100

Esta fragmentación de la información nos permite tener fragmentos lo suficientemente largos para conservar el contexto semántico y no sobrecargar al sistema cuando se analicen. El solapamiento nos garantiza que no haya pérdidas de información entre los cortes, algo importantísimo cuando los datos que manejamos contienen múltiples campos como los metadatos o explicaciones técnicas como los resúmenes.

Esta división fragmentada es clave en la arquitectura RAG ya que los sistemas de recuperación trabajan a nivel de fragmento y no de documentos completos.

B) Embeddings semánticos

Tras generar los fragmentos, el siguiente paso es transformarlo en vectores mediante un modelo de embedding Para realizar con éxito este paso se ha empleado el modelo "j*inaai/jina-embeddings-v2-base-es*" disponible en HuggingFace, a través de la interfaz "*HuggingFaceEmbeddings",* ya que es uno de los modelos que mejor resultado ofrecen para la representación vectorial de textos en español .

Se trata de un modelo entrenado con un corpus de más de 250 millones de documentos en español, lo que garantiza que los vectores generados sean capaces de representan correctamente el contexto del sistema. El resultado de esta transformación es una serie de vectores de dimensiones fijas que representan el contenido y el significado de cada fragmento. Esto nos permite comparar los diferentes fragmentos de una forma efectiva mediante comparaciones de similitud.

Cada uno de los fragmentos previamente creados se transforman en un vector gracias al modelo de embeddigns "*jinaai/jina-embeddings-v2-base-es*". Este modelo está entrenado en español a través de HuggingFaceEmbeddings

C) Indexación en base de datos vectorial

Los vectores previamente creados se almacenan en ChromaDB, una base de datos vectorial optimizada para tareas de búsqueda semántica. Es aquí donde se indexan los embeddings, permitiendo búsquedas por similitud con métricas como la distancia coseno o Euclídea. Este sistema usa un recuperador, o *retriever* con los parámetros:

- Search_kwargs = {“k”: 5}

Esto significa que, ante cualquier consulta del usuario, el sistema recuperará los 5 fragmentos más similares y representativos semánticamente. Esta fase es el corazón del *retrieval* del sistema RAG, selecciona del conocimiento existente lo más representativo y con más relación con la consulta del usuario.

D) Preparación del prompt

Cuando se recuperan los fragmentos más relevantes, se construye la entrada que recibirá el modelo, también llamada *prompt.*

El prompt está estructurado con PromptTemplate, permitiendo definir un formato fijo con campos variables. En este caso el prompt empieza con la asignación del rol seguido de:

- El contexto, que combina los fragmentos recuperados junto con los metadatos del TFG (título, autor, director, ...).
- La pregunta que corresponde a la consulta en lenguaje natural lanzada por el usuario.

```
self.prompt = PromptTemplate.from_template(
    """
    Eres un asistente experto en búsqueda de información en TFGs.
    Usa el siguiente contexto para responder la pregunta.
    Responde de manera concisa

    Contexto: {context}

    Pregunta: {question}

    Si no sabes la respuesta, di que no lo sabes.

    Respuesta:
    """
)
```

Figura 7 Prompt empleado para interactuar con el usuario

Este formato garantiza que el modelo no sufra “alucinaciones” y se centre única y exclusivamente en la información que le proporciona el sistema de recuperación.

La división entre contexto recuperado y pregunta o consulta es lo que da sentido a la arquitectura RAG, proporcionando al modelo generativo un input personalizado y contextualizado.

E) Generación de respuesta

Por último, la generación de respuestas se realiza con el modelo StableLM2, desplegado localmente en el servidor de Ollama. La elección de este modelo se debe

a su capacidad de trabajar en español, su eficiencia computacional y su posibilidad de ejecución en local, garantizando una mayor privacidad y control sobre la información que manejamos.

Se ha definido una temperatura de 0.0, lo que nos asegura que, ante una misma pregunta y contexto, siempre se va a generar la misma respuesta. Esto es clave en sistemas pregunta-respuesta donde la consistencia y repetibilidad son clave para la fiabilidad.

El modelo coge el *prompt*, ya estructurado, y devuelve una respuesta en lenguaje natural con el conocimiento contextual recuperado y la consulta del usuario. Este paso final es lo que completa la lógica RAG, recuperar y generar, en dos etapas separadas pero colaborativas.

Todo el flujo sigue la filosofía del Retrieval-Augmented Generation:

1. Recupera el contenido relevante (*retrieval*) mediante los embeddings y ChromaDB.
2. Genera una respuesta (*generation*) sobre esta base de conocimiento sin depender de la memoria preentrenada del modelo.

Cada componente del sistema tiene su razón de ser:

- La división en fragmentos garantiza eficiencia y contexto suficiente y relevante.
- Los *embeddings* codifican el significado de cada parte.
- ChromaDB posibilita las búsquedas rápidas y relevantes.
- El *prompt* organiza la información para que el modelo la reciba de forma clara.
- StableLM2 transforma el conocimiento en una respuesta comprensible para el usuario en lenguaje natural.

Este diseño modular asegura tanto un funcionamiento correcto como una base sólida para futuras ampliaciones o mejoras.

3.3. Módulo de evaluación

El último módulo, **evaluación,** se encarga de realizar las pruebas al modelo para comprobar su correcto funcionamiento y comprobar su robustez y precisión. Para ello, este módulo incluye:

- Conjunto de preguntas reales formuladas sobre los TFGs almacenados en el JSON, diseñadas para cubrir diversos niveles de dificultad y distintas áreas de información: autores, títulos, grados, materias, etc.
- Las respuestas reales esperadas que sirven como referencia, extraídas directamente del archivo JSON. Sirven para comparar las respuestas que dé el modelo con estas respuestas.
- La llamada al sistema RAG para obtener las respuestas generadas por el modelo.

- La evaluación automática con métricas BERTScore, comparando las respuestas esperadas con las respuestas generadas por el modelo mediante representaciones semánticas profundas.

Conjuntos de evaluación

Se han definido dos conjuntos a evaluar para medir la escalabilidad y robustez del sistema y del modelo:

- Un conjunto reducido con la información de 6 TFGs (tfgs_pequeño.json). Sirve parar validar el correcto funcionamiento inicial del sistema con un volumen de datos reducido.
- Un conjunto completo con información de 25 TFGs (tfgs.json). Utilizado para observar el comportamiento del sistema con un aumento significativo en el número de documentos y la variedad semántica.

Las preguntas realizadas han sido diseñadas para ser variadas y cubrir diferentes niveles de dificultad. Algunos ejemplos de esto son:

- Preguntas simples (fáciles):
  - ¿Cuál es el título del TFG escrito por Mengling Yang?
  - ¿Qué grado cursó Sofía Díaz Fernández?
  - ¿Qué ODS aborda el TFG de Jorge Mora Collado?
- Preguntas intermedias:
  - ¿Qué asignaturas trata el TFG dirigido por María Gema Bello?
  - ¿Cuá es el departamento del TFG relacionado con KNIME y el mercado financiero?
  - ¿Qué palabras clave se asocian al TFG de Díaz Fernández, Javier?
- Preguntas complejas (difíciles):
  - ¿Qué TFG trata sobre segmentación de tumores con modelos de difusión?
  - ¿Cuál es el TFG más relacionado con la acción climática y el medio ambiente?
  - ¿Qué TFG combina redes neuronales con sensores inerciales para ayudar a personas con movilidad reducida?

Esta diversidad en las preguntas asegura que el sistema es evaluado de manera equilibrada, poniendo a prueba su comprensión de distintas partes del contenido y su capacidad para responder cualquier tipo de preguntas, directas y más analíticas.

4. Evaluación y resultados

Se ha llevado a cabo una fase de evaluación centrada en comprobar la capacidad del sistema para recuperar la información de manera precisa y ser capaz de generar respuestas coherentes y relevantes en lenguaje natural, a fin de comprobar que el sistema desarrollado cumple con los objetivos planteados.

4.1 Diseño de la experimentación

La evaluación del sistema al completo se ha basado en comprobar que el sistema RAG implementado es capaz de realizar consultas sobre los TFGs almacenados en los archivos JSON. Para ello, se han creado un conjunto de preguntas relacionadas con los diversos metadatos y resúmenes almacenados en el JSON, observando así la capacidad del modelo para generar respuestas coherentes y adecuadas.

Se han utilizado como muestras dos archivos JSON, uno con la información de 6 TFGs y otro con la información de 25 TFGs, para validar el correcto funcionamiento del sistema. Estas pruebas abarcan casi todos los metadatos incluidos en la información de los TFG como puede ser el autor, grado, materia, ODS, departamento...

Para que la evaluación de la calidad de las respuestas del sistema fuese objetiva y medible se ha utilizado la métrica **BERTScore**. Es una herramienta basada en modelos de lenguaje preentreandos que se encarga de comparar la similitud semántica entre textos. Esta herramienta nos permite evaluar si las respuestas generadas transmiten el mismo significado que la respuesta que esperamos, incluso si esta no es idéntica palabra a palabra. A diferencia de las métricas habituales usadas en PLN, que se centran en comprobar el parecido léxico entre las respuestas y que tienden a penalizar cambios sutiles en la respuesta devuelta por el modelo (por ejemplo, si se usa un tiempo verbal distinto), esta métrica ofrece una mayor flexibilidad, siendo más robusta a este tipo de cambios.

BERTScore calcula tres métricas clave:

- **Precisión (P)**: qué porcentaje de las palabras de la respuesta generada coinciden semánticamente con la respuesta real de referencia.

$$\text{Precision} = \frac{\text{Relevant retrieved instances}}{\text{All } \mathbf{retrieved} \text{ instances}}$$

- **Recall (R)**: qué porcentaje de palabras de la respuesta de referencia están reflejadas en la respuesta generada por el modelo.

$$\text{Recall} = \frac{\text{Relevant retrieved instances}}{\text{All } \mathbf{relevant} \text{ instances}}$$

- **F1 Score**: media armónica entre Precisón y Recall, dando una medida del equilibrio entre ambas.

$$F_1 = \frac{2}{\mathrm{recall}^{-1} + \mathrm{precision}^{-1}}$$

Procedimiento

El proceso de evaluación se estructuró del siguiente modo:

1. **Diseño de preguntas:** se definieron 10 preguntas sobre los metadatos de los TFGs. Buscamos abarcar diferentes niveles de dificultad y variedad semántica con preguntas como:
    a. Consultas directas sobre el **autor**, como: "¿Cuál es el TFG escrito por CheCui?"
    b. Preguntas por el **departamento**, por ejemplo: "¿Qué TFG ha sido desarrollado en el Departamento de Sistemas Informáticos?"
    c. Preguntas sobre **grados académicos**: "¿Qué TFG corresponde al Grado en Ciencia de Datos e Inteligencia Artificial?"
    d. Consultas sobre las **ODS (Objetivos de Desarrollo Sostenible)**: "¿Qué TFG está relacionado con la ODS número 13 sobre Acción por el Clima?"
    e. Preguntas sobre **temáticas o materias específicas**: "¿Qué TFG aborda la temática de accesibilidad?"

    Este conjunto de preguntas fue diseñado para comprobar la correcta recuperación de información, la capacidad de comprender el contexto, la capacidad de cruzar distintos metadatos y responder a diversas consultas que mezclan distintos elementos del contenido.

2. **Generación de respuestas:** se ejecutaron las consultas en el sistema con los dos conjuntos de datos disponibles, el pequeño (6 TFGs) y el grande (25 TFGs).
3. **Comparación semántica:** se compararon las respuestas generadas por el sistema con las respuestas reales obtenidas de la información del JSON original.
4. **Evaluación cuantitativa:** mediante bert-score se calcularon las métricas de Precisión (P), Recall (R) y F1 Score, siendo esta la más representativa.

4.2 Resultados obtenidos

Los valores obtenidos para la muestra menor de información de TFGs, el JSON con información de 6 TFGs distintos, fue:

- **Precisión: 0.6527**
- **Recall: 0.7611**
- **F1 Score: 0.7010**

Los valores obtenidos para la muestra mayor de información de TFGs, el JSON con información de 25 TFGs distintos, fue:

- **Precisión: 0.5853**
- **Recall: 0.6933**
- **F1 Score: 0.6328**

	Muestra menor (6 TFGs)	Muestra mayor (25 TFGs)
Precisión	**0.6527**	**0.6047**
Recall	**0.7611**	**0.6677**
F1 Score	**0.7010**	**0.6338**

Tabla 1: Resumen de los resultados obtenidos por el modelo

Los resultados anteriores muestran que el sistema es capaz de recuperar información y generar respuestas en lenguaje natural con gran precisión. El valor obtenido en la métrica F1 Score con el conjunto pequeño de datos fue de 0.7010 y con el conjunto grande de 0.6328. Ambos valores en estas métricas se consideran positivos dentro del contexto de las tareas pregunta-respuesta, reflejando una gran similitud semántica entre las respuestas generadas por el modelo y las esperadas.

Aunque los resultados no son perfectos, hayamos resultados muy positivos para un sistema que trabaja con datos reales y abiertos y que utiliza un modelo generativo ligero como es StableLM2, un modelo optimizado para funcionar en local.

La diferencia entre ambos resultados puede deberse a:

- **Mayor ruido y ambigüedad** en el conjunto grande, 25 TFGs, introduciendo mayor variabilidad en los metadatos y dificultando una precisa y eficiente recuperación de estos.
- **Limitaciones del modelo StableLM2**, aun siendo un modelo generativo que funciona en español puede experimentar problemas a la hora de manejar grandes cantidades de datos con un contexto similar.
- **La longitud y el formato** de las respuestas generadas por el modelo, en algunos casos pueden introducir información innecesaria o desordenada haciendo que las métricas disminuyan su precisión.

Pregunta realizada	Respuesta esperada	Respuesta generada
¿Qué TFG trata sobre la segmentación de tumores cerebrales mediante modelos de difusión?	Diffusion Model Based Brain Tumor Segmentation Enhance With Inpainting Method.	El Título del TFG que trata sobre la segmentación de tumores cerebrales mediante modelos de difusión es "Diseño e implementación de un sistema para la segmentación y análisis de tumor cerebrovascular". En este proyecto, se busca desarrollar una solución informática que permita a los neurocirujanos segmentar y analizar el comportamiento del agua en las diferentes áreas cerebrales de un paciente con tumor

		cerebral. La aplicación de modelos de difusión es fundamental para lograr esta tarea, ya que permite la visualización y análisis de grandes conjuntos de datos de imágenes cerebrales.
¿Quién es el autor del TFG sobre control de una silla de ruedas usando redes neuronales?	Díaz Fernández, Sofía.	No se ha identificado a ningún autor específico del TFG sobre el control de una silla de ruedas usando redes neuronales en los metadatos disponibles.
¿Qué TFG se centra en el desarrollo de un sistema de análisis de datos geográficos y medioambientales?	Desarrollo de un sistema para el análisis de datos geográficos y temporales sobre el medioambiente.	En este TFG se centra en el desarrollo de un sistema de análisis de datos geográficos y medioambientales. No se ha encontrado ninguna coincidencia exacta en los metadatos para identificar nombres de autores específicos, pero el proyecto aborda la manipulación y análisis de grandes conjuntos de datos geográficos y medioambientales a través del uso de herramientas como Python y plataformas especializadas.
¿Cuál es el ODS relacionado con el TFG de Carmen Miralles Martín?	03. Salud y bienestar, 11. Ciudades y comunidades sostenibles, 13. Acción por el clima.	El ODS relacionado con el TFG de Carmen Miralles Martín es "Salud y bienestar".
¿Qué asignaturas están relacionadas con el TFG ‘Sistema de gestión de un restaurante’?	Informática.	En cuanto a las asignaturas relacionadas con el TFG 'Sistema de gestión de un restaurante', podrías incluir materias como: 1. Informática: Desarrollo de sistemas informáticos, diseño y programación de aplicaciones. 2.. 3. Sistemas Operativos: Implementación y mantenimiento de sistemas operativos (Windows, Linux, etc.). 4. Redes y Hacks: Conocimientos en seguridad informática y técnicas de red para la gestión de un restaurante. 5. Programación Desarrollo Web: Diseño y desarrollo web con lenguajes como HTML, CSS y JavaScript. 6. Gestión de Proyectos: Técnicas y herramientas para el desarrollo de proyectos en el ámbito del restaurante.
¿En qué fecha fue defendido el TFG	Marzo 2025.	No se ha encontrado una coincidencia exacta en los metadatos correspondiente

‘Diseño e implementación de una red neuronal para el manejo de una silla de ruedas’?		al TFG 'Diseño e implementación de una red neuronal para el manejo.. Para encontrar información específica sobre la fecha de defensa del TFG, puedes consultar los registros académicos de la Universidad Pública donde se imparte dicha materia.
¿Qué TFG está relacionado con el uso de la plataforma KNIME en el análisis financiero?	Prototipo de modelo predictivo implementado en KNIME Analytics Platform basado en aprendizaje profundo para la toma de decisiones en el mercado de valores financieros.	No existe un TFG específico relacionado con el uso de la plataforma KNIME en el análisis financiero dentro del contexto proporcionado. Sin embargo, este proyecto podría ser relevante para ti si tu intención es analizar y procesar datos financieros utilizando herramientas avanzadas como KNIME. Para lograr esto, podrías utilizar esta plataforma en combinación con otras herramientas de análisis y visualización de datos, como Excel o Python.
¿Quién dirigió el TFG ‘Diseño y programación de una aplicación web para la reserva de laboratorios’?	Bordel Sánchez, Borja.	El director que dirigió el TFG 'Diseño y programación de una aplicación web para la reserva de laboratorios' fue Alberto Díaz Álvarez.
¿Qué palabras clave se asocian al TFG de Che Cui sobre segmentación de tumores?	Segmentación de tumor; Imágenes médicas; Supervisión débil; Traducción imagen a imagen; Modelos de difusión; Restauración de imagen; Inpainting; Denoising Diffusion Probabilistic Models (DDPM).	En el TFG de Che Cui sobre segmentación de tumores, las palabras clave que se asocian son "segmentación", "tumores", y "computadora". No hay coincidencia exacta en los metadatos con respecto a otras palabras clave.
¿Qué TFG aborda la personalización de modelos para usuarios con movilidad reducida usando Raspberry Pi?	Diseño e implementación de una red neuronal para el manejo de una silla de ruedas.	No lo he encontrado en los metadatos de los documentos relacionados con el caso de estudio "Conflicto Israelí Palestino". Sin embargo, puedo informar que la tarea TFG aborda la personalización de modelos para usuarios con movilidad reducida usando Raspberry Pi. El objetivo es desarrollar un software utilizando esta plataforma y herramientas de IA

		(Inteligencia Artificial) como base, con el fin de adaptar y mejorar los servicios y experiencias ofrecidos a personas con necesidades específicas en relación a su movilidad. En resumen, la tarea TFG busca explorar y aplicar técnicas de IA para desarrollar soluciones personalizadas y adaptativas que satisfagan las necesidades y expectativas de usuarios con movilidad reducida.

Estas preguntas y resultados corresponden a las pruebas realizadas al conjunto con mayor cantidad de información, consiguiendo los resultados de las métricas mencionados anteriormente.

4.3 Conclusiones

La evaluación de las métricas demuestra, de manera objetiva y fiable, que el sistema **funciona correctamente** y es capaz de generar respuestas contextualizadas y acertadas con los datos disponibles. Es importante comentar que el rendimiento disminuye ligeramente cuando el volumen de datos aumenta, pero se sigue manteniendo en unos valores correctos y aceptables. Esto demuestra que la arquitectura basada en RAG implementada en el sistema es **robusta y escalable**.

Como adelanto del siguiente apartado, una mejora futura consistiría en integrar un modelo de mayor capacidad para aumentar la precisión de este y, por consiguiente, las métricas generadas.

5. Conclusiones y trabajos futuros

5.1 Conclusiones

En el presente Trabajo de Fin de Grado se ha logrado desarrollar con éxito un sistema basado en la técnica de Retrieval-Augmented Generation (RAG) que permite realizar consultas en lenguaje natural sobre la información de los TFGs defendidos en la ETSISI de la Universidad Politécnica de Madrid. El principal objetivo del sistema es facilitar el acceso a la información académica, de una forma más intuitiva, eficiente y precisa ya que actualmente no contamos con herramientas que nos faciliten este tipo de tareas.

A lo largo del desarrollo de este proyecto se han conseguido diferentes logros significativos:

- Se ha implementado un proceso de scrapping automatizado con Selenium, capaz de extraer toda la información de los TFGs publicados en oa.upm.es y estructurando esta información en un archivo JSON estandarizado.
- Gracias a HuggingFace se ha llevado a cabo una indexación semántica con embeddings en español, esto nos permite representar los metadatos de los TFGs de una forma más precisa.
- Se ha desarrollado y desplegado un sistema RAG funcional en un entorno local, gracias a LangChain, ChromaDB y el modelo generativo StableLM2 del servidor Ollama.
- El sistema desarrollado responde, de una manera correcta y contextualizada, consultas complejas y variadas sobre los metadatos almacenados en los archivos JSON.

Para cerrar el proyecto, el sistema ha sido evaluado mediante las métricas BERTScore, comparando las respuestas generadas por el modelo con las respuestas reales esperadas. La evaluación ha sido llevada a cabo con dos conjuntos de datos:

- El primero de estos conjuntos tiene la información relativa a 6 TFGs distintos, obteniendo una puntuación de 0.7010 en F1 Score, demostrando una alta precisión del modelo en escenarios con un volumen de datos bajo.
- El conjunto más extenso, 25 TFGs distintos, obtuvo un 0.6328 en la métrica F1 Score, esto indica que sigue siendo un sistema efectivo incluso aumentando el volumen de datos. Esta ligera pérdida de precisión puede deberse al aumento de complejidad y ambigüedad en las respuestas generadas.

En resumen, el sistema ha cumplido satisfactoriamente su objetivo principal y, ha demostrado que es un sistema robusto, funcional y escalable, permitiendo recuperar información de TFGs de una forma sencilla para cualquier tipo de usuario.

5.2 Trabajos futuros

Tras conocer los múltiples resultados satisfactorios del proyecto surgen también una serie de mejoras que podrían ser abordadas en futuras versiones como:

1. **Ampliación de la base de datos:** actualmente el sistema cuenta con un número limitado de TFGs. Automatizar la extracción de información permitiría mantener la información actualizada y cubrir todos los TFGs disponibles en el repositorio.
2. **Uso de modelos con mayor capacidad:** si sustituimos el modelo actual, StableLM2, por otro más potente como puede ser Llama 3, Phi-2 o incluso un modelo multimodal ayudaremos a mejorar la generación de respuestas y su coherencia y exactitud en contextos con más información donde buscar o ambiguos.
3. **Incorporación de respuestas multilingües:** si extendemos la funcionalidad con el fin de poder realizar consultas y generar respuestas en otros idiomas, inglés, sobre todo, aumentaríamos la accesibilidad del sistema para estudiantes internacionales o investigadores extranjeros.
4. **Interfaz de Usuario (UI):** Crear e integrar una aplicación web con interfaz gráfica de usuario hará que el sistema sea más usable, como requisito no funcional, ya que permitirá su uso a usuarios que no tengan conocimientos técnicos. Esto ampliará aún más el alcance del proyecto.
5. **Optimización del procesamiento:** incluir técnicas de re-ranking de respuestas o filtrado por metadatos ayudarán a mejorar la precisón del modelo a la hora de generar una respuesta más coherente, mejorando también sus métricas.
6. **Incluir más métricas de evaluación:** al incluir nuevas métricas como puede ser BLEU o Rouge, incluso evaluaciones huamnas, para complementar la validación del sistema.
7. **Integración con el buscador oficial de la UPM:** podría conectarse este sistema con el portal oa.upm.es para complementar y mejorar la búsqueda avanzada de TFGs desde el propio portal oficial de la universidad.
8. **Mayor personalización:** permitir al usuario filtrar resultados por año, grado, ODS o palabras clave, por ejemplo.

Como conclusión, este proyecto sienta las bases para una solución real y aplicable dentro del entorno académico, con diferentes formas de expansión en lo técnico y funcional. Este proyecto no solo aborda una necesidad concreta de accesibilidad a cierta información académica, sino que también representa una oportunidad para aplicar técnicas de inteligencia artificial en un contexto útil y tangible para todo tipo de usuarios, estudiantes, investigadores, docentes, etc.

6. Sostenibilidad e impacto social

El desarrollo de este sistema no solo representa un avance tecnológico sino también una contribución importante en la sostenibilidad y el impacto social. Con el correcto funcionamiento del sistema conseguimos reducir la brecha de conocimiento dentro de la comunidad universitaria, promoviendo un acceso intuitivo y eficiente al conocimiento.

Uno de los aspectos más destacables en relación con los Objetivos de Desarrollo Sostenible (ODS) definidos por la Organización de las Naciones Unidas (ONU):

- ODS 4 - Educación de calidad: el sistema desarrollado mejora la accesibilidad a cualquier recurso educativo, ayudando a los estudiantes, docentes e investigadores a consultar este tipo de información de una forma más eficiente, impulsando su proceso formativo y aprendizaje.
- ODS 9 – Industria, Innovación e Infraestructura: la implementación de tecnologías avanzadas como este sistema RAG, modelos generativos y bases de datos vectoriales, entre otros, fomenta la constante innovación y el desarrollo de infraestructuras digitales inteligentes en el ámbito académico.
- ODS 10 - Reducción de las desigualdades: al suprimir las barreras técnicas en el acceso a este tipo de información, documentos académicos, contribuimos a reducir las desigualdades en el acceso a la información, beneficiando a aquellos usuarios con menos experiencia tecnológica.
- ODS 3 – Salud y bienestar: algunos TFGs incluidos en la base de datos tratan temas relacionados con la salud, accesibilidad o tecnologías asistiva. El sistema permite resaltar y difundir este tipo de trabajos, promoviendo la sensibilización y el desarrollo de soluciones sociales centradas en el bienestar de las personas.

Además, el desarrollo del sistema se ha realizado con un enfoque de bajo consumo computacional, ejecutándose de forma local y utilizando modelos ligeros, minimizando así el impacto energético frente a otras soluciones basadas en servicios en la nube. Esto convierte a este sistema en una herramienta más sostenible desde el punto de vista medioambiental.

En conjunto, el proyecto demuestra un uso responsable de la Inteligencia Artificial alineado con objetivos sociales, medioambientales y de sostenibilidad, integrando tecnologías innovadoras y avanzadas con un enfoque humanista y accesible.

Bibliografía

Asai, A., Schick, T., Lewis, P., Chen, X., Izacard, G., Riedel, S., Hajishirzi, H and Yih, W. (2023).Cross-Document Reasoning for Multi-Hop QA. Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, 15044-15057.

Bojanowski, Piotr (2017). Enriching Word Vectors with Subword Information. En: arXiv preprint arXiv: 1607.04606. URL: https://arxiv.org/abs/1607.04606

Cho, Kyungghyun (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. En: arXiv preprint arXiv: 1406.1078. URL: https://arxiv.org/abs/1406.1078

Devlin Jacob (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. En: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). Minneapolis, Minnesota: Association for Computational Linguistics, págs 4171-4186. DOI: 10.18653/v1/N19-1423. URL: https://aclanthology.org/N19-1423

Gadekallu, T.R., Alazab, M., Maddikunta, P.K.R., Pham, Q.V., Reddy, M.P. and Liyanage, M. (2023). Generative Pre-trained Transformer: A Comprehensive Review on Enabling Technologies, Potential Applications, Emerging Challenges and Future Directions. ArXiv preprint arXiv:2305.10435. https://arxiv.org/abs/2305.10435

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, M. y Wang, H. (2024) A survey on Retrieval-Augmented Generation. ArXiv. https://doi.org/10.48550/arXiv.2312.10997

IBM (2023). Retrieval Augmented Generation – Architectures. Recuperado de https://www.ibm.com/architectures/patterns/genai-rag

Jeffrey Pennington Richard Socher, Chrostopher Manning (2014). GloVe: Global Vectors for Word Representation. En: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). Doha, Qatar: Association for Computational Linguistics, págs 1532-1543. DOI: 10.3115/v1/D14-1162. URL: https://aclanthology.org/D14-1162

Johnson J., Douze, M., Jégou, H. (2022) Vector Database Optimization Techniques for Large-Scale Retrieval. Proceedings of the 2022 ACM SIGMOD International Conference on Management of Data, 1123-1136.

KDnuggets (2025). Semantic Search with Vector Databases. Recuperado de https://www.kdnuggets.com/semantic-search-with-vector-databases

Khare, Y., Bagal, V., Mathew, M., Devi, A., Priyakumar, U. D. and Jawahar, C. (2021). Mmbert: Multimodal bert pretraining for improved medical vqa. In 2021 IEEE 18th International Symposium on Biomedical Imaging (ISBI), págs 1033-1036. IEEE.

Lai, J., Gan, W., Wu, J., Qi, Z. and Yu, P. S. (2024). Large language models in law: A surbey. AI Open, 5:181-196.

Latard, B., Weber, J., Forestier, G., Hassenforder, M. (2017). Towards a Semantic Search Engine for Scientific Articles. In: Kamps, J., Tsakonas, G., Manolopoulos, Y., Iliadis, L., Karydis, I. (eds) Research and Advanced Technology for Digital Libraries. TPDL 2017. Lecture Notes in Computer Science(), vol 10450. Springer, Cham. https://doi.org/10.1007/978-3-319-67008-9_54

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Roktäschel, T., Riedel, S. and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M. and Li, H., editors, Advances in Neural Information Processing Systems, volume 33, págs 9459-9474. Curran Associates, Inc.

Li, X. and Li, J (2024). Angle-optimized text embeddings.

Liu, R., Jia, C., Wei, J., Xu, G., Wang, L. and Vosoughi, S. (2021). Mitigating political bias in language models through reinforced calibration. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 35, págs 14857-14866.

Mezhlumyan, A. (2024). Building a RAG System for Academic Papers (Technical Report No. CS-2024-01). American University of Armenia College of Science and Engineering.

Mikolov, T., Chen, K., Corrado, G. and Dean, J. (2013). Efficient Estimation of Word Representation in Vector Space. arXiv preprint arXiv: 1301.3781. URL: https://arxiv.org/abs/1301.3781

OpenAI (2023). GPT-4 Technical Report. En: arXiv preprint arXiv:2303.08774. URL: https://arxiv.org/bas/2303.08774

Patel, H. (2024). Bane and boon of hallucinations in context of generative ai.

Ramesh, Aditya (2021). Zero-Shot Text-to-Image Generation. En: arXiv preprint arXiv:2102.12092. URL: https://arxiv.org/abs/2102.12092

Shen, Y., Heacock, L., Elias, J., Hentel, K. D., Reig, B., Shih, G. and Moy, L. (2023) Chatgpt and other large language models are double-edge swords.

Thulke, D., Daheim, N., Dugast, C. and Ney, H., (2021) Multi-Step Factual Consistency Checking for Abstractive Summarization. Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics, 3301-3311.

Turing, A. M. (2009). Computing machinery and intelligence. Springer.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017). Attention is all you need. Advances in Neuronal Information Processing Systems, 30, 5998-6008.

https://papers.nips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al (2023a). A survey of large language models. arXiv preprint arXiv:2303.18223